

Proiect de Programare / C++

== Sistem de management al angajaților ==

Student 1: Mihai

Student 2: Bogdan

Plan tehnic al proiectului

I. Enunț și împărțirea taskurilor

Proiectul reprezintă o aplicație C++ pentru managementul angajaților unei întreprinderi. Sistemul permite administratorului să introducă, să modifice, să șteargă, să caute și să afișeze angajații. Datele fiecărui angajat includ: ID, nume, adresă, post ocupat, data angajării și salariu.

Aplicația va folosi mecanisme de manipulare a fișierelor pentru preluarea și salvarea datelor. Informațiile citite din fișiere vor fi încărcate în structuri de date C++ bazate pe clase și pe colecții de obiecte.

Pentru a permite lucrul în paralel, proiectul va fi împărțit în două executabile care folosesc aceleași fișiere de date și aceleași clase comune:

- **Aplicația 1 - app_1.exe:** administrarea angajaților - responsabil Mihai.
- **Aplicația 2 - app_2.exe:** căutare, afișare și raportare - responsabil Bogdan.

Taskuri Mihai - administrare date

- Implementarea executabilului app_1.exe.
- Implementarea operațiilor de adăugare, modificare și ștergere angajat.
- Generarea automată a ID-ului pentru angajații noi.
- Validarea datelor introduse: format dată, salariu pozitiv, câmpuri obligatorii.
- Implementarea funcțiilor de salvare în fișier după operațiile de modificare.
- Scrierea în fișierul de log pentru operațiile care schimbă datele.

Taskuri Bogdan - căutare și raportare

- Implementarea executabilului app_2.exe.
- Implementarea operațiilor de afișare a tuturor angajaților.
- Implementarea căutării după nume, după post și după nume + post.
- Implementarea filtrării după interval salarial.
- Implementarea rapoartelor: număr total de angajați, angajați grupați pe posturi, medie salarială.
- Exportarea rezultatelor de căutare/raportare într-un fișier text.

Taskuri comune

- Definirea claselor comune: Angajat, DataCalendaristica, EmployeeRepository, CommandParser.
- Stabilirea formatului fișierului comun angajati.csv.
- Testarea integrării între cele două executabile.
- Code review și verificarea finală a comenzilor expuse.

II. Structura datelor folosite de echipă

Vom folosi clase C++ pentru a înlocui structura procedurală din varianta inițială în C. Datele vor fi păstrate în memorie într-un vector<Angajat>, iar citirea/scrierea se va face prin clase specializate.

Clasă	Descriere
DataCalendaristica	Reține data angajării: int zi, int luna, int an. Conține metode pentru validarea formatului zz/ll/aaaa, conversia din string și conversia în string.
Angajat	Reține datele unui angajat: int id, string nume, string adresa, string post, DataCalendaristica dataAngajare, double salariu. Conține metode pentru afișare și serializare în format CSV.
EmployeeRepository	Gestionează colecția de angajați și comunicarea cu fișierul angajati.csv. Conține metode precum load(), save(), findById(), getAll(), getNextId().
EmployeeService	Conține logica aplicației pentru operații CRUD: addEmployee(), updateEmployee(), deleteEmployee(). Apelează repository-ul și validatoarele.
SearchService	Conține logica de căutare și filtrare: searchByName(), searchByJob(), searchByNameAndJob(), filterBySalary().
ReportService	Generează rapoarte: număr total de angajați, grupare după post, salariu mediu, export rezultate în fișier.
Validator	Validează datele primite din linia de comandă: dată validă, salariu pozitiv, câmpuri nevidate, ID numeric.
CommandParser	Interpretează argumentele primite prin argc și argv. Identifică numele comenzii, parametrii și afișează mesaje de eroare pentru comenzi invalide.
AuditLogger	Scrie în audit.log operațiile importante: adăugare, modificare, ștergere, export raport.

Relația dintre clase

- app_1.exe folosește EmployeeService, EmployeeRepository, Validator, CommandParser și AuditLogger.
- app_2.exe folosește SearchService, ReportService, EmployeeRepository, CommandParser și AuditLogger.
- Ambele aplicații folosesc aceleași clase Angajat și DataCalendaristica.

III. Structura fișierelor folosite de către echipă

Structura directoarelor proiectului

```
EmployeeManagementCpp/  
|-- data/  
|   |-- angajati.csv  
|   |-- audit.log  
|   |-- raport.txt  
|-- include/  
|   |-- Angajat.h  
|   |-- DataCalendaristica.h  
|   |-- EmployeeRepository.h  
|   |-- EmployeeService.h  
|   |-- SearchService.h  
|   |-- ReportService.h  
|   |-- Validator.h  
|   |-- CommandParser.h  
|   |-- AuditLogger.h  
|-- src/  
|   |-- Angajat.cpp  
|   |-- DataCalendaristica.cpp  
|   |-- EmployeeRepository.cpp  
|   |-- EmployeeService.cpp  
|   |-- SearchService.cpp  
|   |-- ReportService.cpp  
|   |-- Validator.cpp  
|   |-- CommandParser.cpp  
|   |-- AuditLogger.cpp  
|   |-- main_app_1.cpp  
|   |-- main_app_2.cpp  
|-- README.md  
|-- CMakeLists.txt
```

Fișierul principal de date: data/angajati.csv

Acest fișier este folosit pentru comunicarea dintre cele două executabile. app_1.exe modifică datele, iar app_2.exe le citește pentru căutări și rapoarte.

```
<numar_angajati>  
<id>;<nume>;<adresa>;<post>;<data_angajare>;<salariu>  
<id>;<nume>;<adresa>;<post>;<data_angajare>;<salariu>
```

...

Exemplu:

```
3
1;Popescu Andrei;Cluj-Napoca;Programator;12/03/2021;6500.00
2;Ionescu Maria;Bucuresti;Contabil;01/09/2020;5200.50
3;Dumitru Mihai;Oradea;Manager HR;15/01/2019;7800.00
```

Reguli pentru fișier:

- Separatorul dintre câmpuri este caracterul ;.
- Prima linie conține numărul total de angajați.
- Data este păstrată în formatul zz/ll/aaaa.
- Salariul este păstrat ca număr real cu două zecimale.
- ID-ul este unic și este generat automat de app_1.exe.

Fișierul de log: data/audit.log

Acest fișier reține acțiunile importante efectuate în aplicație. Este util pentru verificarea operațiilor și pentru depanare.

```
<data_ora>;<aplicatie>;<actiune>;<id_angajat>;<detalii>
```

Exemplu:

```
23/05/2026 14:20;app_1;ADD;4;Angajat adaugat: Radu Elena
23/05/2026 14:25;app_1;UPDATE;2;Camp modificat: salariu
23/05/2026 14:31;app_2;EXPORT;-;Raport exportat in raport.txt
```

Fișierul de raport: data/raport.txt

Acest fișier este generat de app_2.exe atunci când utilizatorul exportă rezultatele unei căutări sau ale unui raport.

```
SISTEM MANAGEMENT ANGAJATI - RAPORT
Tip raport: <tip_raport>
Data generare: <data_ora>

<id> | <nume> | <post> | <data_angajare> | <salariu>
...
```

IV. Interacționarea cu executabilele

Aplicațiile vor fi rulate din linia de comandă. Fiecare comandă validează argumentele primite, citește datele din fișiere, execută operația cerută și afișează un mesaj de succes sau eroare.

Aplicația 1 - app_1.exe - administrarea angajaților

```
./app_1.exe help
```

Afișează lista comenzilor disponibile pentru aplicația de administrare.

```
./app_1.exe vizualizare_angajati
```

Afișează toți angajații existenți în fișierul `angajati.csv`.

```
./app_1.exe vizualizare_angajat <id>
```

Afișează detaliile unui angajat identificat prin ID.

```
./app_1.exe adaugare_angajat "<nume>" "<adresa>" "<post>" <data_angajare> <salariu>
```

Adaugă un angajat nou. ID-ul este generat automat.

Exemplu:

```
./app_1.exe adaugare_angajat "Popescu Andrei" "Cluj-Napoca" "Programator" 12/03/2021 6500.00
```

```
./app_1.exe modificare_angajat <id> <camp> "<valoare_noua>"
```

Modifică un câmp al angajatului. Câmpurile acceptate sunt: nume, adresa, post, data_angajare, salariu.

Exemple:

```
./app_1.exe modificare_angajat 2 salariu "5700.00"  
./app_1.exe modificare_angajat 2 post "Contabil Senior"
```

```
./app_1.exe stergere_angajat <id>
```

Șterge angajatul cu ID-ul specificat.

```
./app_1.exe validare_fisier
```

Verifică dacă fișierul `angajati.csv` respectă structura stabilită.

Aplicația 2 - app_2.exe - căutare și raportare

```
./app_2.exe help
```

Afișează lista comenzilor disponibile pentru aplicația de căutare și raportare.

```
./app_2.exe afisare_angajati
```

Afișează toți angajații, în format tabelar.

```
./app_2.exe cautare_nume "<fragment_nume>"
```

Caută angajații după nume sau după o parte din nume. Căutarea nu este sensibilă la litere mari/mici.

Exemplu:

```
./app_2.exe cautare_nume "popescu"
```

```
./app_2.exe cautare_post "<fragment_post>"
```

Caută angajații după post sau după o parte din denumirea postului.

Exemplu:

```
./app_2.exe cautare_post "programator"
```

```
./app_2.exe cautare_num_e_post "<fragment_num_e>" "<fragment_post>"
```

Caută angajații care respectă simultan criteriul de nume și criteriul de post.

```
./app_2.exe filtrare_salariu <salariu_minim> <salariu_maxim>
```

Afișează angajații care au salariul în intervalul specificat.

Exemplu:

```
./app_2.exe filtrare_salariu 4000 7000
```

```
./app_2.exe raport_posturi
```

Afișează câți angajați există pentru fiecare post.

```
./app_2.exe raport_salariu_mediu
```

Afișează salariul mediu al tuturor angajaților.

```
./app_2.exe export_raport <tip_raport> "<fisier_output>"
```

Generează un raport în fișierul indicat. Tipurile acceptate sunt: `toti`, `posturi`, `salariu_mediu`.

Exemplu:

```
./app_2.exe export_raport posturi "data/raport.txt"
```

V. Fluxul de lucru între cele două aplicații

1. Mihai implementează și testează `app_1.exe`, care modifică fișierul comun `data/angajati.csv`.
2. Bogdan implementează și testează `app_2.exe`, care citește `data/angajati.csv` și generează rapoarte.
3. După fiecare operație de modificare, `app_1.exe` salvează datele în fișier și scrie în `audit.log`.
4. `app_2.exe` reîncarcă datele din fișier la fiecare rulare, astfel încât vede automat modificările făcute de `app_1.exe`.
5. Rezultatele exportate de `app_2.exe` sunt scrise în `data/raport.txt` sau în fișierul primit ca argument.